

KatanaPerps A-1

Security Audit

March 11th, 2026

Version 1.0.0

Presented by [OxMacro](#)

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for KatanaPerps's smart contract code as found in the section titled 'Source Code'. The security audit was performed in two parts by the Macro security team. Part 1 was conducted from January 19th to January 26th, 2026. The second part was conducted from February 11th to February 16th, 2026.

The purpose of this audit is to review the source code of certain KatanaPerps Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Low	8	-	3	5
Informational	1	-	-	-

KatanaPerps was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions on Slack with the KatanaPerps team.
- Documentation files contained in the repo.

Source Code

The following source code was reviewed during the audit:

- **Repository:** [katana-perps-contracts-niseko](#)
- Commit Hash of Part 1 (Diff Audit + Index Price Adapter): 2ac8f8ed5806a23a2486ac64166836b9c5601623
- Commit Hash of Part 2 (Layer Zero Integration): 5995ab6b61af12f7f60cfb38c6fb0463be3a8b3a

For part 1, we audited the diff between commits 4b15268 and 2ac8f8e for the following contracts, as well as a full review of the index price adapters.

Source Code	SHA256
contracts/Custodian.sol	b9002eb21864ecf6f911a568ec445961caeefe225864a764a8a897f7a98992be
contracts/EarningsEscrow.sol	8ab84ac517a135f52c38c8ece5c7e3908669251a54b37586ecbd9c9855aca6e0
contracts/Exchange_v1.sol	6f7f596145232284df7db1dbcf485c5a4c99f6e613b5a4330315d35a159090f8
contracts/Governance.sol	0ff6506329c713a5174ab0aea998e8983376ec6d1e3df006efa8041622f4b09b
contracts/Owned.sol	c9c73bbd0f7959c87285772bf9e46c1e5d7aecba905446644f3d10c7d998124a
contracts/index-price-adapters/ChainlinkDataStreamsIndexPriceAdapter.sol	0ad653925c4ce99a7ab0800a85f41beac99d5eb946fe85fb195ea800bccc604d

contracts/index-price-adapters/KatanaPerpsIndexAndOraclePriceAdapter.sol	38c5cc734465cb2470874dfe3fcc1920f6e8229d268b57e0a4af5bda47f7280f
contracts/index-price-adapters/RedStoneIndexPriceAdapter.sol	36bfb4faa3965498d8fd9a56e7d9ab027b1f4423d2757e3bdded28420442a67a
contracts/libraries/Address.sol	f64dc1cd808c0091ce995772f486ff8af8d2ef38935baedcea652d3248629145
contracts/libraries/AssetUnitConversions.sol	344a1cc1939eab99cabbbe73343b3756b662ec3e212920f6926a9c347bc69bf7
contracts/libraries/BalanceLoading.sol	57c10af02af1eebf84da0e16c1208e30d6ac5dd1fe756248ddbba982654b9025
contracts/libraries/BalanceTracking.sol	a3bb7dd936e9a17f054d859996acc96e87a2b08b929fe73cdd75e893fd18cf08
contracts/libraries/ClosureDeleveraging.sol	40ecd08fe99fcda8f9f6d1e3377d02b082efb86474a6d6848f46c456f2b578b2
contracts/libraries/Constants.sol	f7b557f11f7cf5fad174e92a038b6438bfb8ee15c057d066a6da3ebd4e85df8c
contracts/libraries/Depositing.sol	f1462fbb2874ad3f0b52545ee58f1aa1b2cfb40feb5891c3f89959b17f390938
contracts/libraries/Enums.sol	ebeb139bf937871f951449a3c1c96b9a43f009c122bc49d9721ccfee19be8cf1
contracts/libraries/ExchangeErrors.sol	ec26be9a89199328f52be7bc60342e48a07832b4581e0d114fa6184eb52f1b49

contracts/libraries/ExchangeEvents.sol	ceda48b8661bf993752fd05813e7f48af87a25d218afe732efa455ad83d89f7b
contracts/libraries/ExitFund.sol	09beccdba9bf3824cf147092cb8c25e11e6edd9531f7fe558ab9a99321cd90c2
contracts/libraries/Funding.sol	ee9bc35d7f371bd536c29e777f79192acd55fd4a8251d4fc155c93a71a5f3383
contracts/libraries/FundingMultiplierQuartetHelper.sol	62a2c57f810ef8c21c0af703d6f4ee6bce21eff57de8d3ba66207968d7e8a3de
contracts/libraries/Hashing.sol	6bbb02f9396b3cf719dbfd95642d2cf87fe1327831eec901ff92d4ba2ce9806e
contracts/libraries/IndexPriceMargin.sol	757ebcbe59121636dbb843859ef3d4eb121960f98ae3760ab25b3bd6fbd87b16
contracts/libraries/Interfaces.sol	913205a9c2e12395322b710f4b2ada60d73c6aa0b5b4ee9b2ab74f9458a4b5fc
contracts/libraries/LiquidationValidations.sol	ba8a4734d3250dc46c90ad0592a2e4d47a1979a15878ef95351b3bafbcae7b10
contracts/libraries/ManagedAccounts.sol	ed8a7667e782b97234ed2fa3d8492122a629f8908f6c41e5aa82dea7f4d410b5
contracts/libraries/MarketAdmin.sol	b1da3a0c07ac89eb4b17d1a4d39ebd3a0ac65604424b7e2416aa8c118ec4a70f
contracts/libraries/MarketHelper.sol	ad85cf26d38f962608209352d088df2084573c973e6be776b0717b53445053d7
contracts/libraries/Math.sol	cf33dc3dc314b0e6525e467ddc09bbaf2a51dcbbe56132650aa108c08ddad96

	fa
contracts/libraries/NonceInvalidations.sol	0c6262cae45da84a8a9a45d130f203d297947eeda952e42b11f4020f94978ab9
contracts/libraries/OraclePriceMargin.sol	fc8c72d456117dd2f5b92c137174f9996ce7c166c778b004db84bb89753a33ef
contracts/libraries/PositionBelowMinimumLiquidation.sol	4165387015f78870090c7a162b49dd1031ae7b68735e32bb575540f4a554f36f
contracts/libraries/PositionInDeactivatedMarketLiquidation.sol	753efb7333f1b80e75000d19b85908ec7e7673380baed873198ad487c0767457
contracts/libraries/SortedStringSet.sol	cc218a524be0b4fbbdf4e168ae3e0e19fd5d249c3638cb87d9b4e4daae5686a0
contracts/libraries/String.sol	8b78b454836a3d132835c8a6cbe3162a75689ba7a64a62dc354bf506b52e189a
contracts/libraries/Structs.sol	fe61103b8c6a6a6f813c21e1c6efd69696cd68fab614c8ee1d1517d23b11af1d
contracts/libraries/Time.sol	ee798309e50c7171a14f7ca849a5f3246c1f76f7c7cbdc39a34413f696dc6325
contracts/libraries/TradeValidations.sol	ca59df6d85bf52104115345b970d70334bbf38e47d35884c0b173ca8fbea4bae
contracts/libraries/Trading.sol	1ed2a7ea20aaafc32f8d168694fba188768b68b572e39a4a125bdf29cd6b0e02e
contracts/libraries/Transferring.sol	5f7e66fc078ebf51f162b822e8a4d6a53036b5176a603307e85ae0bf4a4c6ba9

contracts/libraries/UUID.sol	284d26454225a43dc5c69a6d77c520c88fbdc0dd9758354fcb24da317b848c7c
contracts/libraries/Validations.sol	95d4b0b9e907b42f967e7ec9f39c0f9354d47943f4ab6e7aeae261ede64b03c5
contracts/libraries/WalletExitAcquisitionDeleveraging.sol	09180d1b790e32dfc3b736949d6dc149f86ff0553412959941afa0d02318c232
contracts/libraries/WalletExitLiquidation.sol	959f149ddc23cef322ea145968f951a68c6ef1697875f68b6177d4ff8436925b
contracts/libraries/WalletExits.sol	952948ac5cbf2d1bd859822d1e335ce3363d823b170214e1bb680b9f3db197ac
contracts/libraries/WalletInMaintenanceAcquisitionDeleveraging.sol	71c9a9e8e1bfda7b054fe81cad9ddf9e2600e16c0936a536b94c1a8370cfd940
contracts/libraries/WalletInMaintenanceLiquidation.sol	02c9bd6e02fb2432af370b628161db51c7286d58070164cd9fbc49b130046dc7
contracts/libraries/Withdrawing.sol	21c8c764695cef9cc84bf8c17bde262bca524bdacf3c7882979307a984d7b7b9
contracts/oracle-price-adapters/ChainlinkOraclePriceAdapter.sol	7ab7e14f8fecb56c2f301c4b5d0ec47e0f43a022ad4a829cd26bde78b2497069

For part 2, we audited the following contracts:

Source Code	SHA256
contracts/bridge-adapters/ExchangeLayerZeroAdapter_v1	659b0e519dd7af05f9863b49378d4974e8e77182c12da57a30367afc02e53f38

.sol	
contracts/bridge-adapters/KatanaPerpsStargateForwarder_v1.sol	bf4c37cc5a774cb22c51e4566edce352af4202faee398ba7637c058d27ce9fcd
contracts/bridge-adapters/LayerZeroFeeEstimation.sol	8c3d5c3c63ca154afa00dcc4a888781783bfc01bdb8737cc2d105181ca4e6ac9
contracts/bridge-adapters/libraries/BridgeAdapterEvents.sol	1fb1cc1c6fd40e7fc1ae6790e9bd2acc28b50e565f2000e62dde083b9ebafdf8
contracts/bridge-adapters/libraries/ExchangeAdapterComposing_v1.sol	93070867ba01aadfa4efe91181063365316664b9c888f772e9c1f9c9f873be1a
contracts/bridge-adapters/libraries/KatanaPerpsStargateForwarderComposing_v1.sol	f476c8732cd9f66acf8db0fd183c45f5ec81a23b607488d9f7df41ab9b4692e7

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

- ⚡ Missing seconds-to-milliseconds conversion in Chainlink adapter
- L-2 Missing price validation for Chainlink and Redstone price adapters
- L-3 Chainlink price adapter doesn't handle fees
- L-4 RedStone price adapter assumes 8 decimals for all feeds
- ⚡ Forwarder `lzCompose` catch block sends funds to `owner()` without attempting wallet identification
- ⚡ `setComposeParameters` bypasses bounds validation for `minimumWithdrawQuantityMultiplier`
- ⚡ ERC-4626 `previewRedeem` / `redeem` return value discrepancy
- ⚡ `withdrawNativeAsset` uses `.transfer()` with 2300 gas stipend
- I-1 Migration only works from old to new exchange

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:
 - How bad things can get (for a vulnerability)
 - The significance of an improvement (for a code quality issue)
 - The amount of gas saved (for a gas optimization)
2. The high/medium/low **likelihood** of the issue:
 - How likely is the issue to occur (for a vulnerability)
3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, <i>or</i> some funds/assets will be lost, <i>or</i> the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

⌕

Missing seconds-to-milliseconds conversion in Chainlink adapter

TOPIC	STATUS	IMPACT	LIKELIHOOD
Conversion	Fixed ↗	Low	Low

In `ChainlinkDataStreamsIndexPriceAdapter`, the `validFromTimestamp` from Chainlink's ReportV3 is in seconds, but it is assigned directly to `timestampInMs` without conversion:

```
return
IndexPrice({
    baseAssetSymbol: market.baseAssetSymbol,
    timestampInMs: SafeCast.toUint64(report.validFromTimestamp),
    price: priceInPips
});
```

Reference: [ChainlinkDataStreamsIndexPriceAdapter.sol#L195-L200](#)

As a result, `timestampInMs` will contain an incorrect value, leading to unexpected behavior in any logic that relies on it.

Remediation to Consider

Multiply by 1000: `timestampInMs = validFromTimestamp * 1000`

L-2 Missing price validation for Chainlink and Redstone price adapters

TOPIC	STATUS	IMPACT	LIKELIHOOD
Validation	Wont Do	Medium	Low

The ChainlinkDataStreamsIndexPriceAdapter's `validateIndexPricePayload` doesn't validate key fields from the ReportV3 struct, such as:

- `report.expiresAt > block.timestamp`
- `report.price > 0`

Similar validation checks are missing in the **RedStoneIndexPriceAdapter**.

As a result, expired or invalid price reports could be accepted, leading to incorrect index prices being used in the protocol

Remediation to Consider

Add meaningful validation checks for both adapters.

RESPONSE BY KATANAPERPS

We ended up with just the validations wrapped into [MarketAdmin's `publishIndexPrices\_delegatecall`](#). There are a few operational limits on this front. There can be delays in dispatching transactions from the off-chain components to the contracts, in which case validating older price updates is actually a requirement. Pricing `> 0` could work, but any such price update would run afoul of the upstream off-chain logic, and practically we would delist markets well before encountering anything close to 0.

L-3 Chainlink price adapter doesn't handle fees

TOPIC

Conversion

STATUS

Wont Do

IMPACT

Medium

LIKELIHOOD

Low

The `ChainlinkDataStreamsIndexPriceAdapter` does not handle verification fees, as noted in the comments that Katana does not currently have a `feeManager` set:

```
// Katana does not have a FeeManager and no funding is needed to verify
// reports. Validate no FeeManager is set as a sanity check
require(verifier.s_feeManager() == address(0), "FeeManager not supported")
```

Reference: [ChainlinkDataStreamsIndexPriceAdapter.sol#L183-L185](#)

If Chainlink enables fees in the future, the adapter will fail to verify reports.

Remediation to Consider

Two options:

1. Redeploy when needed: Keep current implementation and deploy a new adapter with fee support if Chainlink enables fees. Acceptable if potential downtime and governance delay are tolerable.
2. Implement fee handling now: Adapt the implementation to handle fees as shown in <https://docs.chain.link/data-streams/reference/interfaces/ifeemanager>. This requires:
 - A mechanism to fund the contract
 - A withdraw function to recover tokens
 - Optionally, support for both native and ERC20 fee payment options

RESPONSE BY KATANAPERPS

The Chainlink team has assured us that they will not be deploying a

L-4 RedStone price adapter assumes 8 decimals for all feeds

TOPIC	STATUS	IMPACT	LIKELIHOOD
Conversion	Wont Do	High	Low

The `RedStoneIndexPriceAdapter` assumes all RedStone price feeds use 8 decimals, but this is not guaranteed for all feeds. As a result, if a feed with non-standard decimals is added, the price will be incorrectly scaled, leading to wrong index prices.

Remediation to Consider

Either:

- Make `decimals` configurable per market in the `RedStoneMarket` struct (as done in `ChainlinkDataStreamsIndexPriceAdapter`), or
- Document and enforce that only standard 8-decimal feeds are supported

RESPONSE BY KATANAPERPS

Given the predominance of 8-decimal feeds, we opted to make that decimal count an operational requirement for adding new market pricing.

L-5 Forwarder `lzCompose` catch block sends funds to `owner()` without attempting wallet identification

When `compose()` reverts entirely, the user's tokens go to `owner()`. There are scenarios where the `destinationWallet` could be successfully decoded.

Consider the case where `vbUSDC.deposit` inside `_forwardDeposit` reverts due to a paused vault. In this case all user funds from bridging attempts would go to the owner, causing temporary loss of custody and increasing operational burden since affected users need to be identified and refunded.

Remediation to Consider

Trying to decode `destinationWallet` to reduce operational burden and avoid temporary loss of custody.

RESPONSE BY KATANAPERPS

After discussion and consideration, we landed on a slightly adjacent change. The forwarder did not explicitly validate against a 0x0 `destinationWallet`. Our rationale is:

1. A 0x0 `destinationWallet` is definitely an error condition, and the Forwarder's catch block handles this correctly. This is "expected" in that it is user input prone to errors.
2. None of the other granular failures in `_forwardDeposit` are expected conditions. Rather than reason about which tokens to return or whether the destination wallet is valid, we believe it's cleaner and safer to allow the catch to handle it universally.

⚠️ `setComposeParameters` bypasses bounds validation for `minimumWithdrawQuantityMultiplier`

TOPIC	STATUS	IMPACT	LIKELIHOOD
Validation	Fixed ↗	Medium	Low

The `ExchangeLayerZeroAdapter_v1.setMinimumWithdrawQuantityMultiplier` enforces bounds of [90%, 100%]. But `setComposeParameters` sets the same variable without any bounds check. This also applies during deployment, since the constructor routes through `setComposeParameters`. An accidental value of `0` disables slippage protection entirely; a value above 100% makes all withdrawals fail.

Remediation to Consider

Add the boundary checks `<= MAX_MINIMUM_WITHDRAW_QUANTITY_MULTIPLIER` and `>= MIN_MINIMUM_WITHDRAW_QUANTITY_MULTIPLIER` to the `setComposeParameters` function.

⚡ ERC-4626 `previewRedeem` / `redeem` return value discrepancy

TOPIC	STATUS	IMPACT	LIKELIHOOD
Economics	Fixed ↗	Low	Low

In `_forwardWithdrawal`, `previewRedeem` is called to compute `usdcAmount`, which is then used for the Stargate send and the Ethereum direct transfer. However, the actual `redeem` call may return a slightly different (higher) amount. Per EIP-4626, `previewRedeem` "MUST return as close to and no more than the exact amount of assets that would be withdrawn in a `redeem` call in the same transaction." This means `redeem` can return $\geq \text{previewRedeem}$. As a result, any dust difference between `redeem`'s actual return and `previewRedeem`'s estimate accumulates in the forwarder contract.

Remediation to Consider

Use the amount returned by the actual `redeem` call instead of `previewRedeem` for subsequent transfers.

I-8 `withdrawNativeAsset` uses `.transfer()` with 2300 gas stipend

TOPIC	STATUS	IMPACT	LIKELIHOOD
Transfer	Fixed ↗	Low	Low

The `ExchangeLayerZeroAdapter_v1.withdrawNativeAsset` correctly uses `.call{value: quantity}("")`, but the `StargateForwarder` uses `.transfer()` which forwards only 2300 gas. This will fail if the destination is a contract with a non-trivial `receive()` (e.g., a multisig). Since this is admin-only, impact is limited to operational inconvenience.

Remediation to Consider

Use `.call` instead of `transfer`.

I-1 Migration only works from old to new exchange

TOPIC	IMPACT
Informational	Informational *

In `BalanceTracking`, the balance migration logic in `loadBalanceStructAndMigrateIfNeeded` copies balance data from a source Exchange but omits the `managedAccountProvider` field. This is intentional and works for migration from an old exchange (without `managedAccountProvider`

support) to a new exchange. However, if a future Exchange upgrade migrates from a version that already supports `managedAccountProvider`, all managed account associations would be lost.

Document this limitation for future upgrade planning. When migrating from an Exchange that supports managed accounts, the migration logic must be updated accordingly.

RESPONSE BY KATANAPERPS

This is an intentional design decision for the current upgrade path. We don't yet know what target structure we will need to support in any future upgrade. As a result, we plan to update the balance migration logic as part of any upgrade.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the KatanaPerps team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.